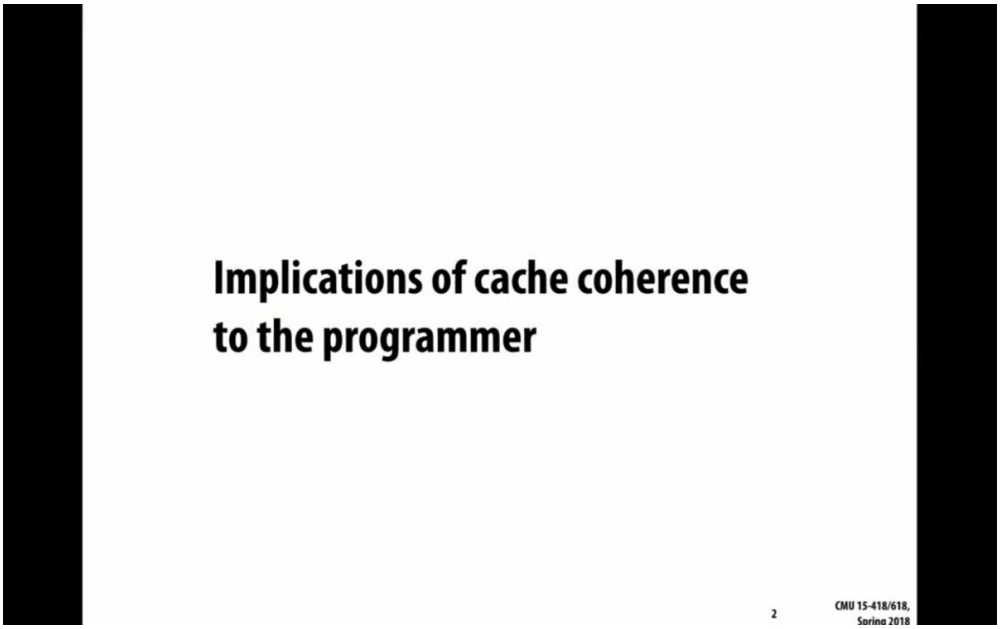
课程笔记

缓存一致性的实现：伪共享、目录协议与内存模型

CMU 15-418/618 并行计算机架构与编程 · Lecture 13（Spring 2018）

lecture-to-notes

2026 年 10 月 28 日



视频作者/频道：CMU 15-418/618（B 站搬运：alexphil）
发布日期：2018 年春季学期
视频时长：1 小时 7 分 0 秒
视频链接：`https://www.bilibili.com/video/BV1wN4116725/`

目录

1 伪共享：代码看起来没问题，性能却崩了 **3**
1.1 机制 . . . 3
1.2 一个能直接看到差别的例子 . . . 4
1.3 与分块布局的关系 . . . 5
1.4 本章小结 . . . 5
1.5 本章小结（续） . . . 6

2 监听协议的扩展性上限 **6**
2.1 问题在哪 . . . 6
2.2 理想与现实的差距 . . . 7
2.3 本章小结 . . . 8

3 目录：把广播变成点对点 **8**
3.1 基本思想 . . . 8
3.2 数据结构 . . . 9
3.3 三种典型情形 . . . 10
3.4 为什么必须等待确认 . . . 11
3.5 home 没有副本的情形 . . . 12
3.6 本章小结 . . . 12

4 目录的收益与代价 **12**
4.1 收益的前提 . . . 12
4.2 实测统计 . . . 13
4.3 访问模式的分类 . . . 14
4.4 本章小结 . . . 14

5 目录的存储代价与压缩方案 **15**
5.1 全位向量的开销 . . . 15
5.2 溢出怎么办 . . . 16
5.3 稀疏目录 . . . 17
5.4 本章小结 . . . 18

6 真实机器的实现与优化 **18**
6.1 转发介入 . . . 18
6.2 Intel Core i7：L3 充当目录 . . . 18
6.3 多插槽系统 . . . 19
6.4 Xeon Phi . . . 20
6.5 本章小结 . . . 21

7 内存一致性模型：一致性没解决的那一半 **21**
7.1 缓存一致性不等于程序直觉 . . . 21
7.2 为什么会允许重排 . . . 22
7.3 怎么写出正确的代码 . . . 22

7.4 布局是最后一项可调的手段 22
7.5 本章小结 22

8 总结与延伸 **23**

1 伪共享：代码看起来没问题，性能却崩了

讲者开场补上第 11 讲没讲完的内容，他说这可能是程序员最需要理解的一件事：伪共享。

1.1 机制

设想有一段数据，其中若干部件恰好落在同一条缓存行里，但被不同核上的不同线程使用。它们开始为这份数据争抢。

假设计算是可写的，例如两个线程各自读写自己的计数器。原则上这些操作完全独立，各自的 L1 缓存可以分别装载这些值，系统本可以跑得很好。

但因为它们恰好被分配在同一条缓存行里，缓存系统认为这相当于要对每个值提供独占访问。于是两个核陷入协议层面的争夺，而写操作的开销远高于读。讲者的原话是：共享完全不可能发生，程序就失控了。

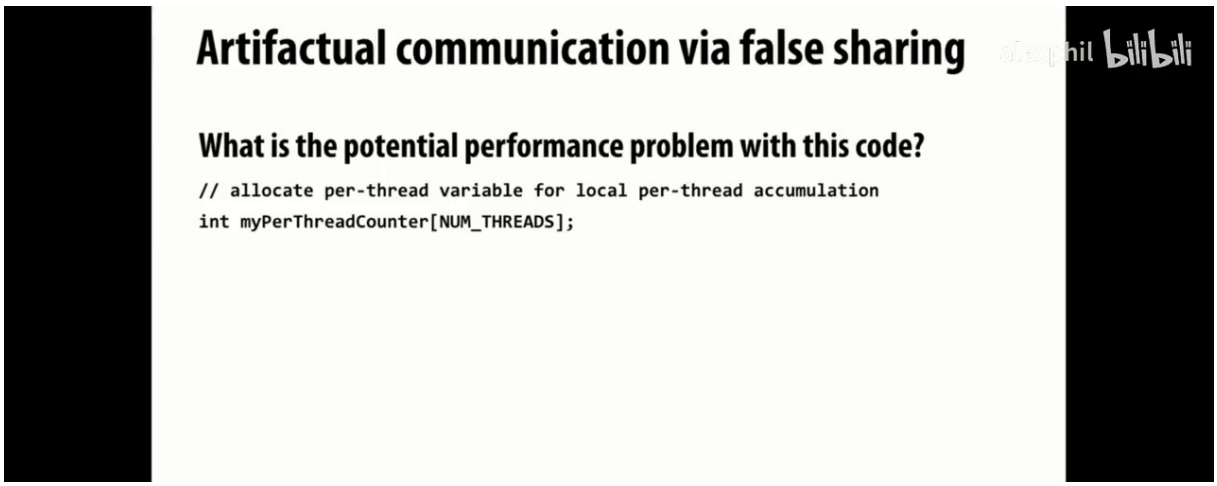


图 1: 人为通信：伪共享造成的额外开销[1]

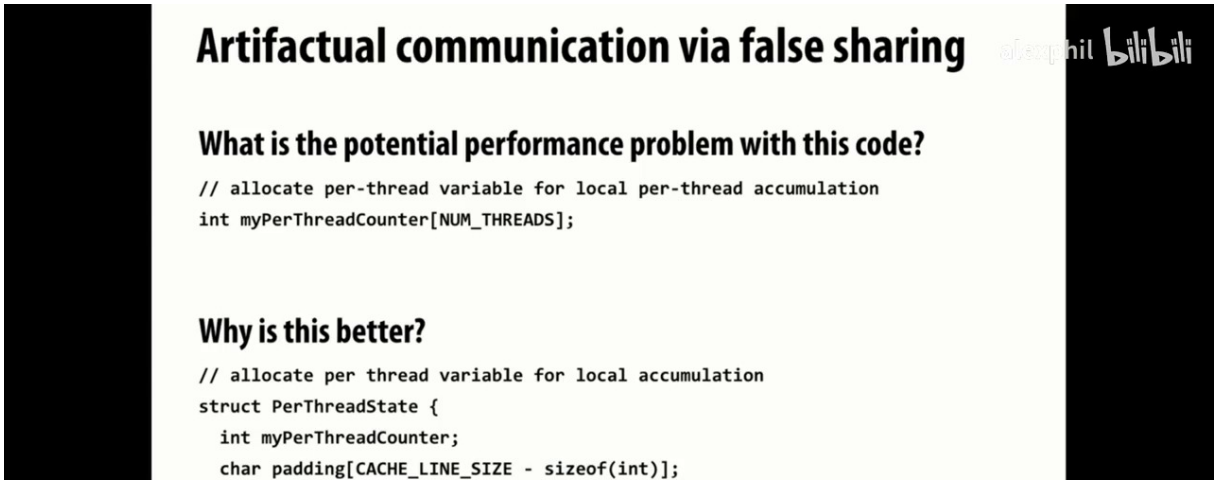


图 2: 伪共享示意：两个处理器写不同地址却争用同一缓存行[2]

[1] 视频画面时间区间：00:01:30–00:01:45。若为裁剪图，时间区间仍对应原视频画面。

[2] 视频画面时间区间：00:03:00–00:03:15。若为裁剪图，时间区间仍对应原视频画面。

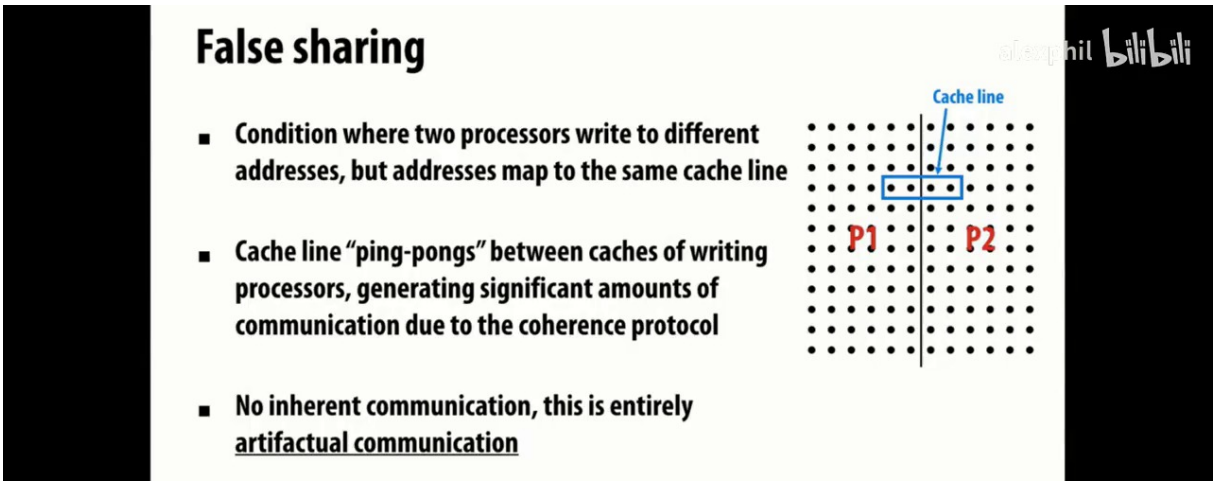


图 3: 伪共享的精确条件：两个处理器写不同地址，但落在同一缓存行[3]

伪共享的判据是缓存行，不是变量

两个线程是否伪共享，取决于它们的写目标是否落在同一缓存行内，与它们是否访问同一个变量无关。一条 64 字节的行可以容纳十六个 int 计数器，若每个线程各用一个，表面上毫无冲突，实际每次写都在抢占同一行的所有权。

1.2 一个能直接看到差别的例子

讲者用 pthreads 举例。分配一个与线程数等长的计数器数组，每个线程独立自增自己那个计数器。

另一个版本做了填充：数组里每个计数器后面跟一些空位，使它们各自落进不同的缓存行。

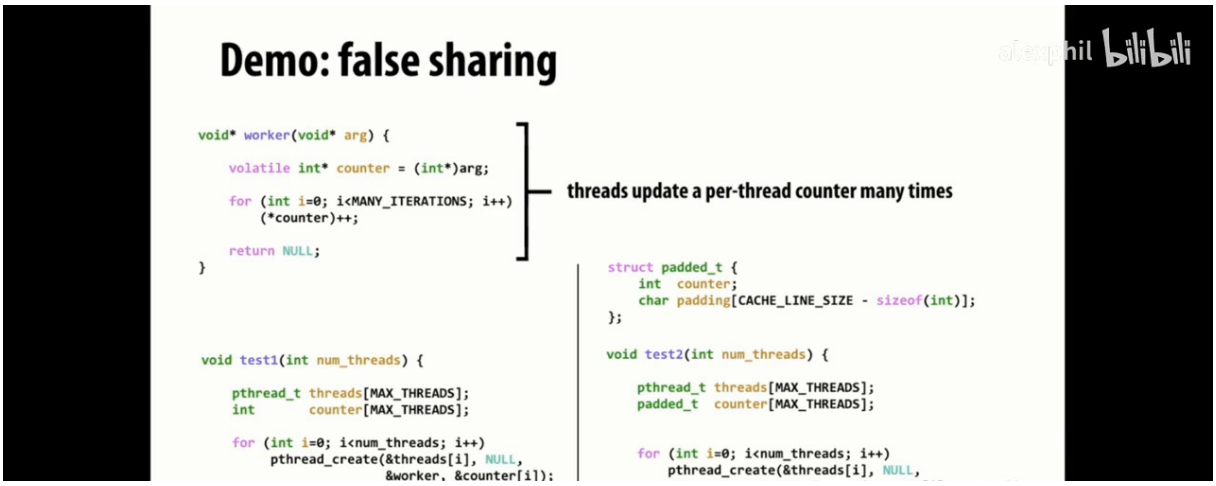


图 4: 伪共享演示：无填充的计数器数组与带填充版本[4]

第一个版本里所有计数器都在同一行，伪共享极其严重；第二个版本各自独占一行，彼此完全独立。

[3] 视频画面时间区间：00:05:00–00:05:15。若为裁剪图，时间区间仍对应原视频画面。

[4] 视频画面时间区间：00:04:30–00:04:45。若为裁剪图，时间区间仍对应原视频画面。

可以把伪共享的代价形式化为一行的往返次数：

$$N_{\text{bounce}} \approx \min(W_1, W_2), \qquad T_{\text{extra}} \approx N_{\text{bounce}} \cdot t_{\text{line}}$$

- W_1, W_2：两个线程各自对该缓存行的写次数；
- N_{bounce}：该行在两次写之间被交接的次数；
- t_{line}：一次缓存行失效加传输的耗时。

填充后 W_2 对应的行与 W_1 不再相同，N_{bounce} 直接降为零。

在一台 12 核机器上的实测是：一种情况 5 秒，另一种 2 秒。讲者强调这已经是相当温和的情形，实际可以差得多。

$$\frac{T_{\text{false sharing}}}{T_{\text{padded}}} = \frac{5}{2} = 2.5$$

- 分子：所有计数器落在同一缓存行时的耗时；
- 分母：每个计数器独占一行时的耗时；
- 比值越大，说明这行数据在核间弹跳得越厉害。

无同步代码也可能被缓存协议拖慢

伪共享的代码里没有任何锁、没有任何同步原语。从并发正确性角度看它完全正确，从性能角度看却在滥用整个缓存系统。这类问题不会报错、不会给出错误结果，只在性能曲线上表现为莫名其妙的退化，因此必须在测量中主动排查。

1.3 与分块布局的关系

讲者指出这正是为什么第 8 讲讨论二维分块时采用那种四维的布局方式：让每个处理器拿到一段连续地址。

这样伪共享最多只发生在这些区段的交界处，影响相对温和，而不会在每一行都发生。

布局把伪共享从”每行”降为”每边界”

若按行分配给不同处理器，那么每行的两端都会与邻居产生共享行，伪共享发生次数与行数同阶。若按块分配，只有块边界处才有共享行，次数与块数同阶。同一个问题，仅仅改变数据归属方式，伪共享的量级就降了一个层次。

1.4 本章小结

伪共享是缓存行粒度造成的：逻辑上独立的变量因同行而互相作废。它不破坏正确性，只破坏性能，实测可以差两倍以上。对策是填充使各线程的写目标独占缓存行，或改变布局让共享只发生在边界。

理解了这个代价，才能理解为什么一致性协议本身的实现方式会直接决定程序性能。下一章转向协议如何扩展。

1.5 本章小结（续）

讲者用一句话收束这段补充：即使没有任何同步，在这些机器上编程也必须理解这类现象。这为整讲定下基调：一致性不是硬件内部的细节，它的实现方式会渗到程序性能里。

2 监听协议的扩展性上限

2.1 问题在哪

讲者先回顾第 11 讲的监听协议：某个缓存发生任何值得关注的操作时，它必须广播给所有其他缓存。

只要同时监听的缓存只有 6 个左右，这没问题。但规模再往上，流量就太多了——总线上的通信本身成为整个系统的中心瓶颈。

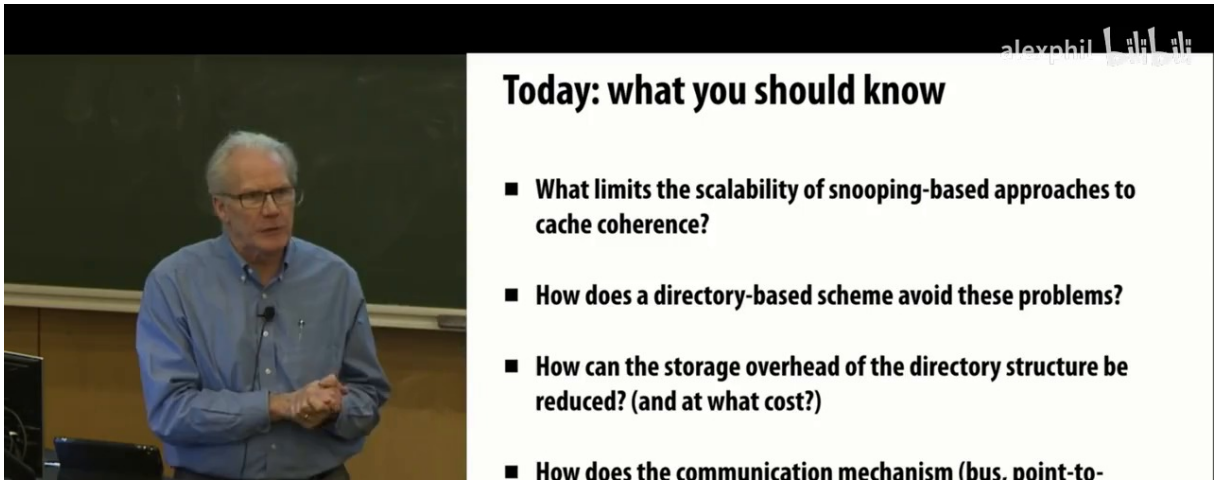


图 5: 本讲要点：什么限制了一致性的可扩展性[5]

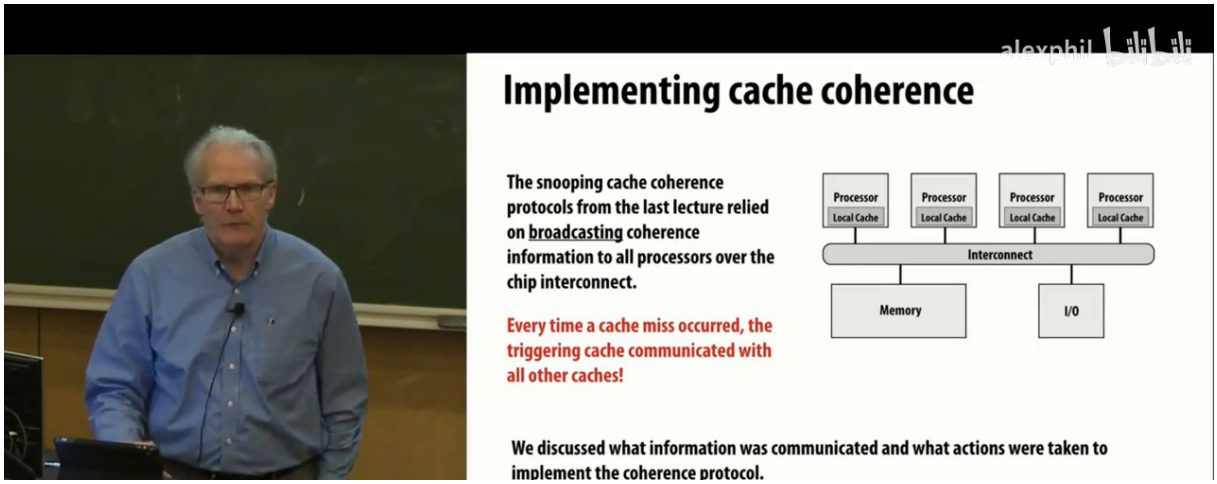


图 6: 实现缓存一致性：监听协议依赖处理器间广播[6]

[5] 视频画面时间区间：00:07:30–00:07:45。若为裁剪图，时间区间仍对应原视频画面。
[6] 视频画面时间区间：00:08:30–00:08:45。若为裁剪图，时间区间仍对应原视频画面。
[7] 视频画面时间区间：00:10:30–00:10:45。若为裁剪图，时间区间仍对应原视频画面。

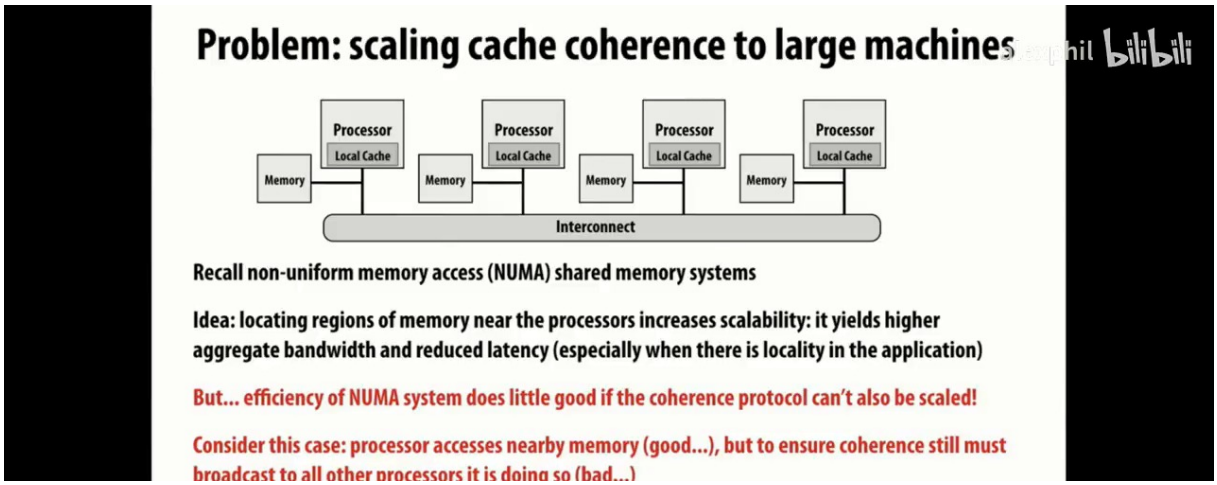


图 7: 问题：把缓存一致性扩展到大型机器[7]

讲者解释总线的含义：它就是一组共享的导线，任何人放上去的数据所有人都会看到并能接收。可以把它想成一个所有消息进出的中心点。

具体来说，一个缓存要访问某数据，就得在总线上发出请求说” 我要这行的独占读或非独占读”；要升级自己的副本为可写，就得广播并作废他人的副本。互连上的流量因此非常重。

2.2 理想与现实的差距

理想方案是点对点：只在持有数据的处理器与想要数据的处理器之间通信。

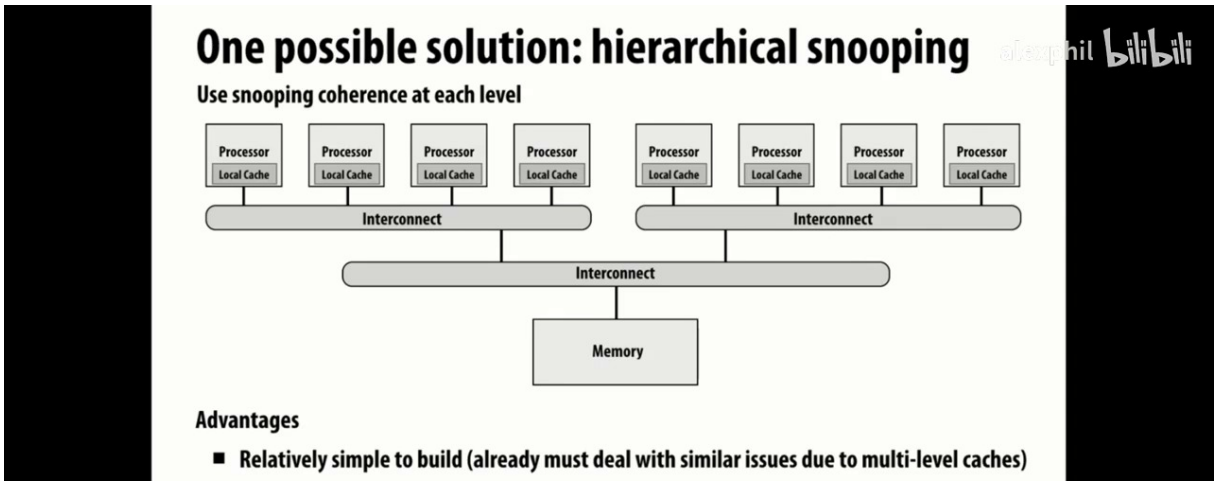


图 8: 一种解法：层次化监听，在每一层使用监听一致性[8]

讲者说现实中没人用简单版本，而是用层次化结构：小规模的例子容易跑，系统变大就叠加更复杂的方案。

一个常见做法是把互连做成树状，让能在局部第一层总线上解决的流量留在那里。比如某处理器要读，而持有者恰好在同一个局部总线上，这次读就在本地解决。若持有者在别处，才需要穿越更高层次的树。

[8]视频画面时间区间：00:13:00–00:13:15。若为裁剪图，时间区间仍对应原视频画面。

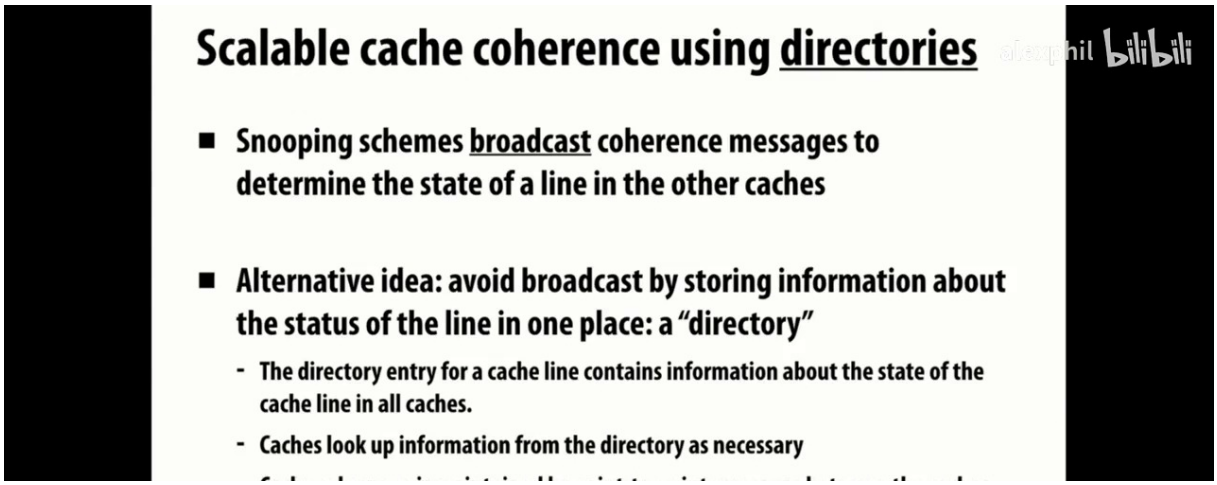


图 9: 用目录实现可扩展的缓存一致性[9]

优点是降低中心拥塞。但讲者明确点出最坏情况：如果工作负载在内存流量上没有被很好地划分，树根就会成为新的瓶颈。

他也提到这种结构在 NUMA 场景下更有效——内存本身被切成多个片段，本地化带来的收益更大。

2.3 本章小结

监听协议的瓶颈是广播：流量随参与者数量增长，超过约半打缓存后互连成为中心瓶颈。层次化监听用树结构把局部流量吸收在低层，降低中心拥塞，但最坏情况下树根仍是瓶颈，除非负载在内存访问上已经划分良好。

要真正把通信变成点对点，需要一份记录”谁持有哪份数据”的信息。这就是目录。

3 目录：把广播变成点对点

3.1 基本思想

讲者设定场景：多个处理器各自能访问内存，内存被分区到各处理器上，希望实现全局的缓存一致访问。做法是引入目录。

目录记录哪些处理器持有某个数据块的副本。有了这份信息，读的时候可以直接去找持有者取，作废的时候只需给持有副本的那些处理器发消息，而不必广播。

[9]视频画面时间区间：00:14:30–00:14:45。若为裁剪图，时间区间仍对应原视频画面。

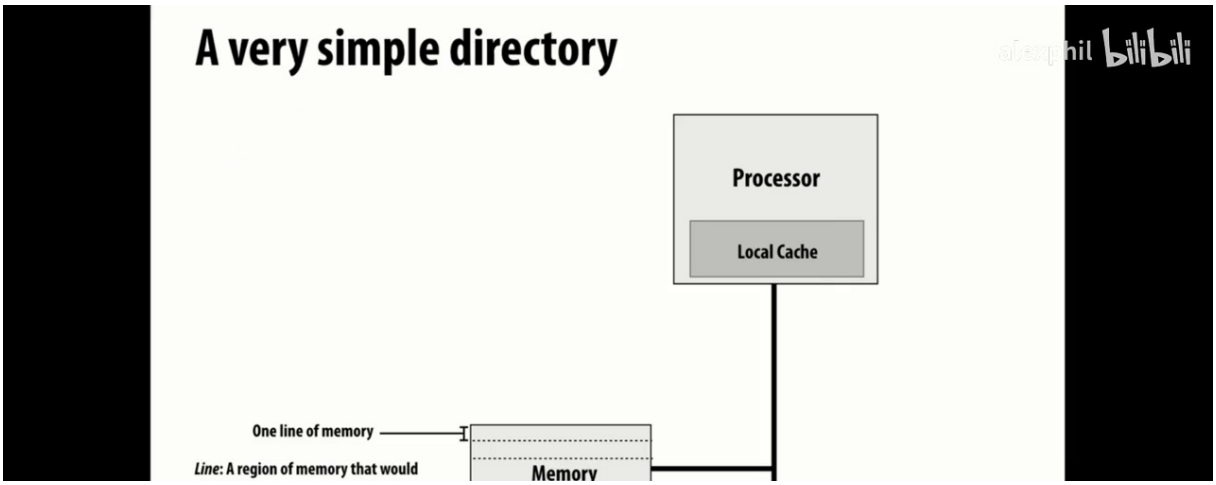


图 10: 一个非常简单的目录：内存按行划分，每行配一组状态位[10]

3.2 数据结构

内存和缓存一样按块组织，这里称为行，为讨论方便取 64 字节一段。

在此基础上，为内存中每一行增加一组位：为系统中每个可能的处理器准备一位，再加一个位表示该行是否脏。这组位向量说明哪些处理器在自己的缓存里有这一行的副本。

为简化，只考虑单级缓存。

与此配套的几个术语：每一行 64 字节的内存在某个处理器上有它的 home node，也就是它在这台机器的内存空间中归属的位置——这个位置与该数据实际在哪被使用完全无关。另外，发起读或写的处理器称为 requesting node。

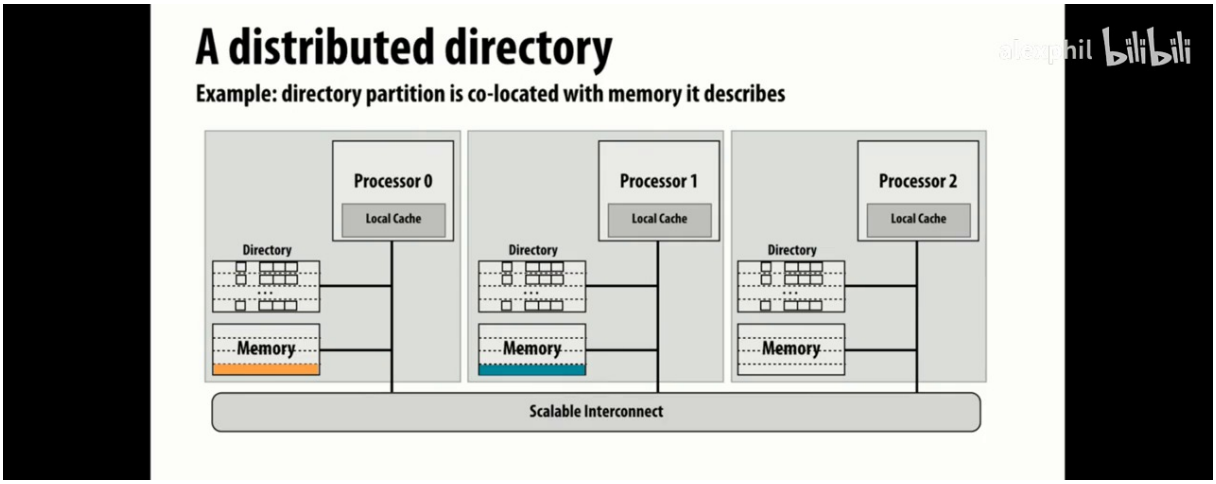


图 11: 分布式目录：目录分区与其描述的内存共置[11]

[10] 视频画面时间区间：00:15:30–00:15:45。若为裁剪图，时间区间仍对应原视频画面。

[11] 视频画面时间区间：00:17:00–00:17:15。若为裁剪图，时间区间仍对应原视频画面。

home node 与持有者是两个不同的概念

home node 由地址决定，是固定的；持有者随程序执行而变化。目录本身存在 home 上，因此请求总是先发往 home，再由 home 决定是直接返回数据，还是把请求转给真正的持有者。混淆这两者就无法理解后面”home 没有副本” 的情形。

3.3 三种典型情形

讲者用三个例子说明协议。假设内存某行由处理器 1 拥有，处理器 0 想读。

情形一：读缺失且该行干净。处理器 0 发现自己没有副本，查地址得知归属处理器 1，于是发出一个读请求。处理器 1 直接回一份副本，并在位向量里标出处理器 0 现在有副本。

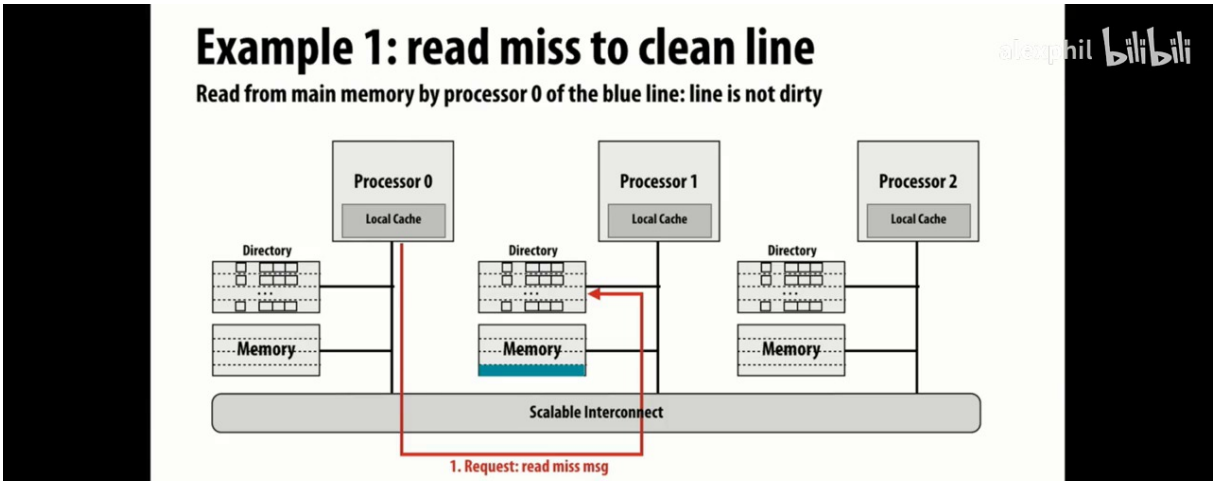


图 12: 例一：读缺失，行为干净，直接由 home 返回数据[12]

情形二：读缺失但该行脏。请求同样发往 home。这时 home 发现自己这份是脏的——或者不是最新副本，需要处理更复杂的情形。

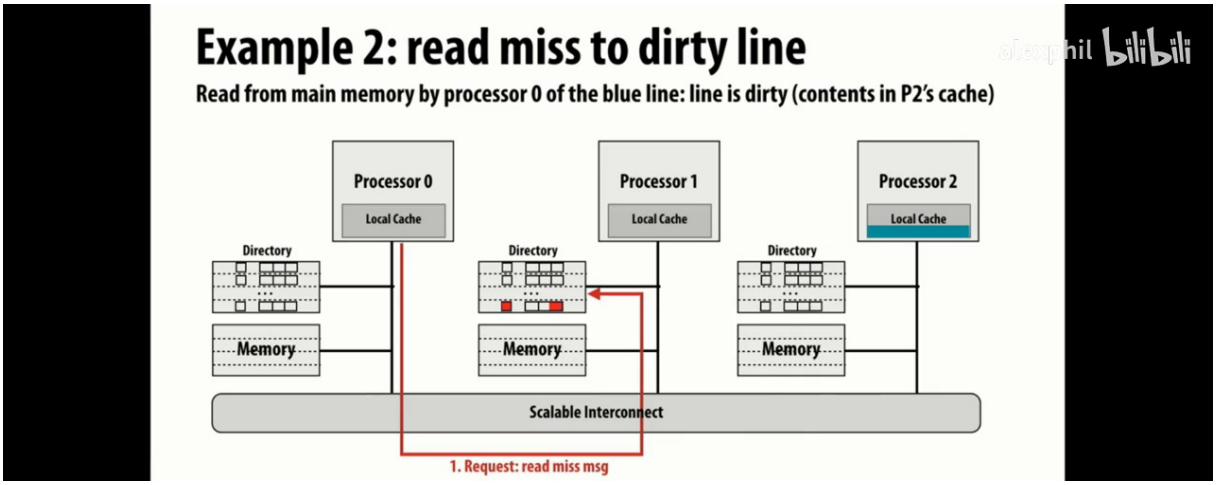


图 13: 例二：读缺失但行为脏，需要先取得最新副本[13]

[12] 视频画面时间区间：00:18:00–00:18:15。若为裁剪图，时间区间仍对应原视频画面。

[13] 视频画面时间区间：00:19:00–00:19:15。若为裁剪图，时间区间仍对应原视频画面。

情形三：写缺失，该行干净但被处理器 1 和 2 持有。这是最复杂的例子。

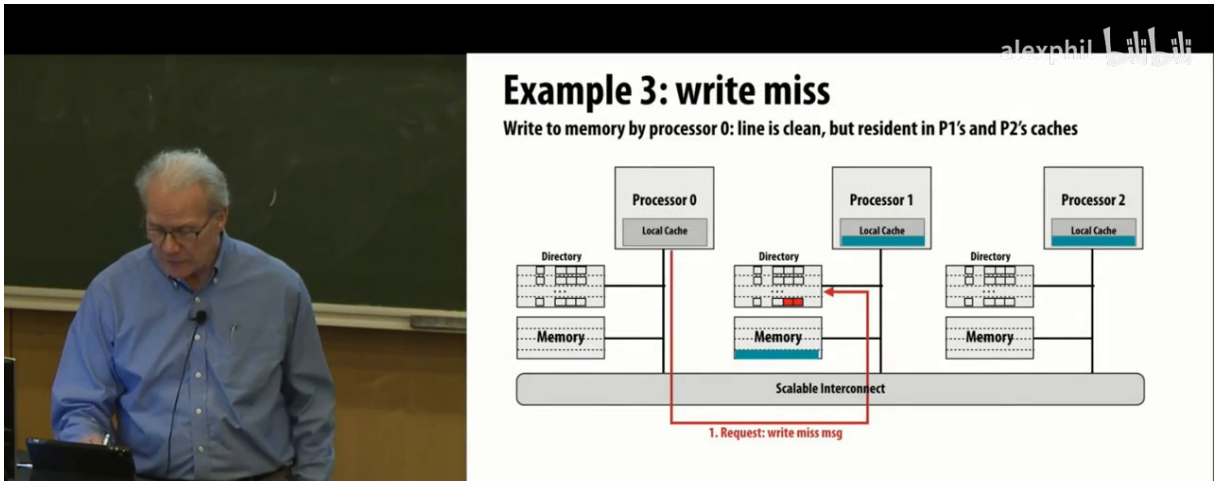


图 14: 例三：写缺失，行干净但驻留在 P1 与 P2 的缓存中[14]

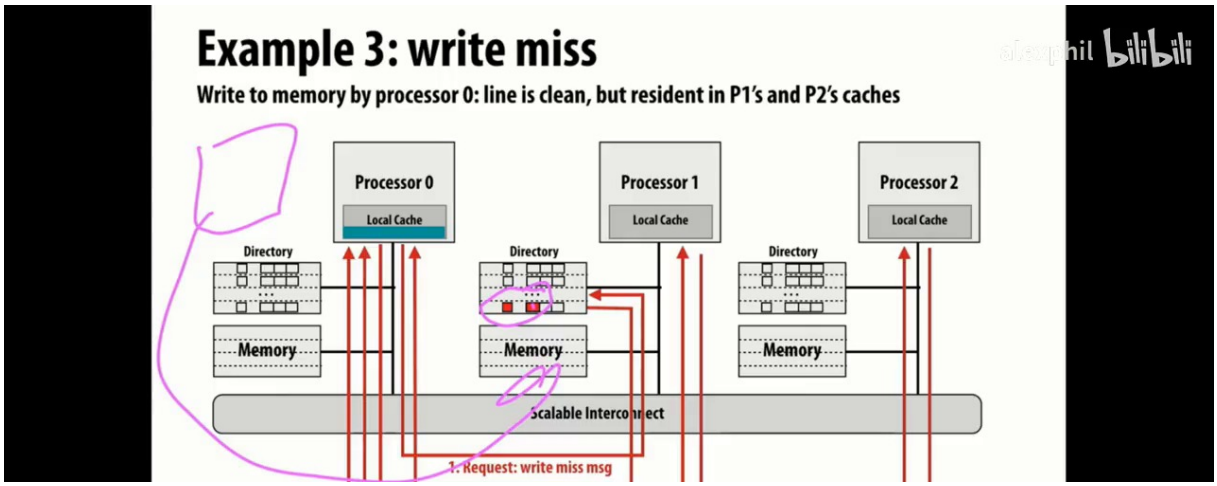


图 15: 例三续：必须等到全部失效确认后才能继续[15]

3.4 为什么必须等待确认

讲者在这里强调了正确性的核心。

前面看过的失效协议里，必须确保失效真的发生了，否则处理器 0 和 1 可能跑在前面，访问一个本该被标记为无效的内存位置。

因此处理器 0 必须等到收到全部确认之后才继续。而这时持有者才把该行标记为脏、持有在处理器 0 的缓存里。

[14] 视频画面时间区间：00:21:00–00:21:15。若为裁剪图，时间区间仍对应原视频画面。

[15] 视频画面时间区间：00:24:00–00:24:15。若为裁剪图，时间区间仍对应原视频画面。

失效确认是正确性的关键，不只是性能细节

没有等待全部 ack 就继续执行，可能出现两个处理器都认为自己持有可写副本的局面，那就是灾难。协议必须保证” 独占” 这个判断在全局唯一，而唯一性靠的是确认链，不是靠时序巧合。这类细节在多核一致性协议中极其复杂，讲者也承认真实协议远比示意版本麻烦。

还有一个细节值得记录：需要适当的 FIFO 队列，保证各处消息不相互越过，否则在等待期间若有其他处理器发来请求，可能得到一个错误的副本。

3.5 home 没有副本的情形

学生提出一个讲者没走到的情形：处理器 0 是请求者，而 home 自己也没有副本。讲者承认这确实是个未覆盖的例子，并分析说这本质上是对脏行的写缺失。

这时 home 无法提供数据，必须增加一步：home 回话说” 其实这行在处理器 2 或 3 的缓存里”。好处是，如果 home 拿不出来，就说明存在一份未共享的可写副本。于是请求者可以直接发消息要求对方交出。

讲者评价说真实协议” 毛发一样多”（极其繁琐），他展示的只是最基础版本。

3.6 本章小结

目录用位向量记录每行的持有者，使通信从广播变成点对点：读时直接找持有者，作废时只通知持有者。正确性依赖失效确认与消息顺序，真实协议的复杂度远超示意版本。

这一章解释了机制，但没说明它是否值得。下一章用实测统计回答这个问题。

4 目录的收益与代价

4.1 收益的前提

讲者给出关键判断：目录减少总线流量的前提是共享者很少。

设想系统有 256 个处理器，但某行只有一份副本在外面。那么只需给那一个发失效消息，而不是广播给 256 个。在这种相当常见的场景下，流量大幅下降。

反过来说，如果共享者很多，目录相对广播协议并不能省什么。

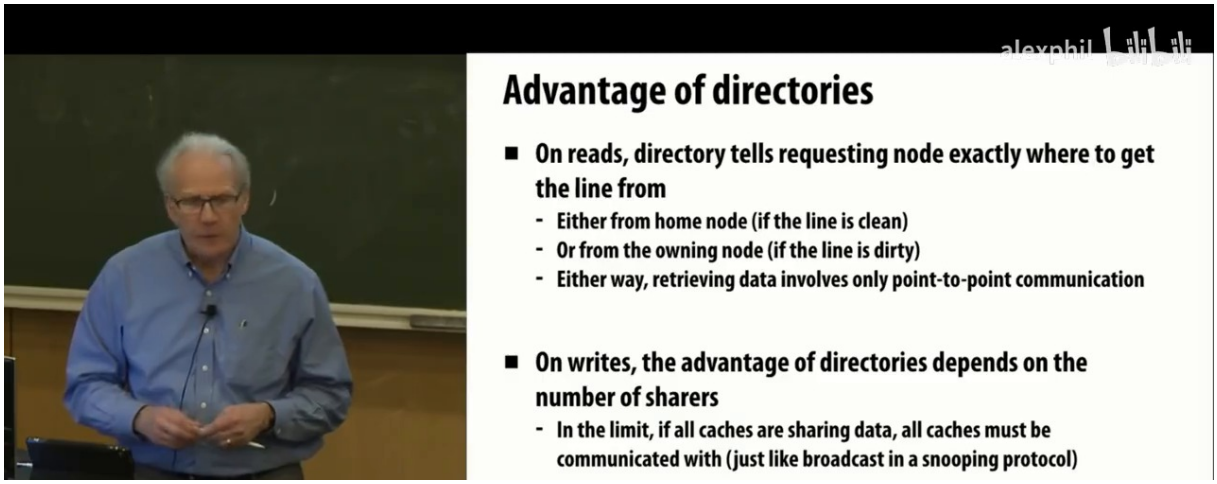


图 16: 目录的优势：读时告知确切来源，失效时只通知持有者[16]

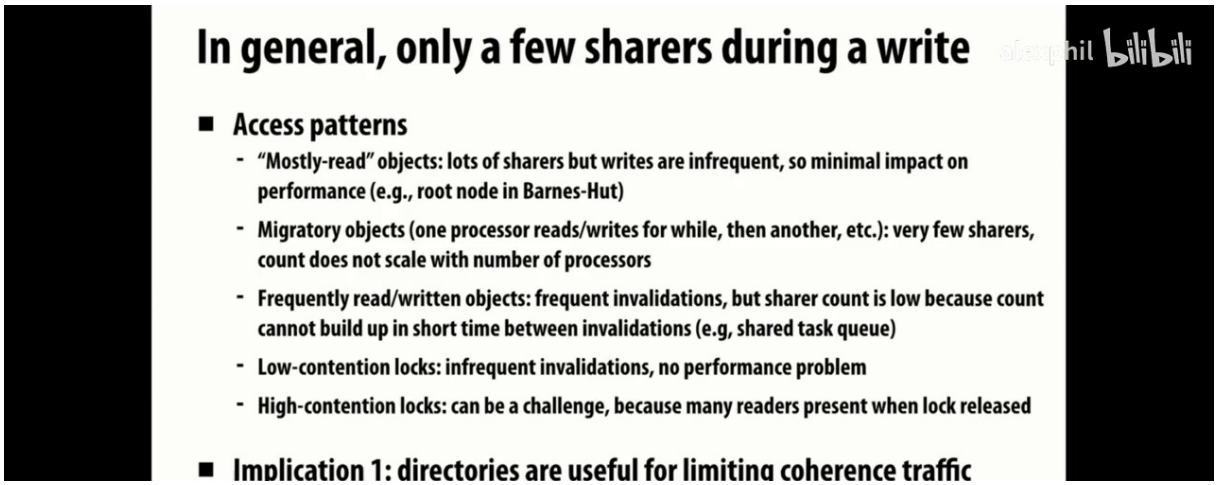


图 18: 统计显示写时通常只有很少共享者[18]

4.2 实测统计

讲者引用了一本教科书里的统计数据：在若干基准程序上，统计每次写时该行被多少处理器共享。

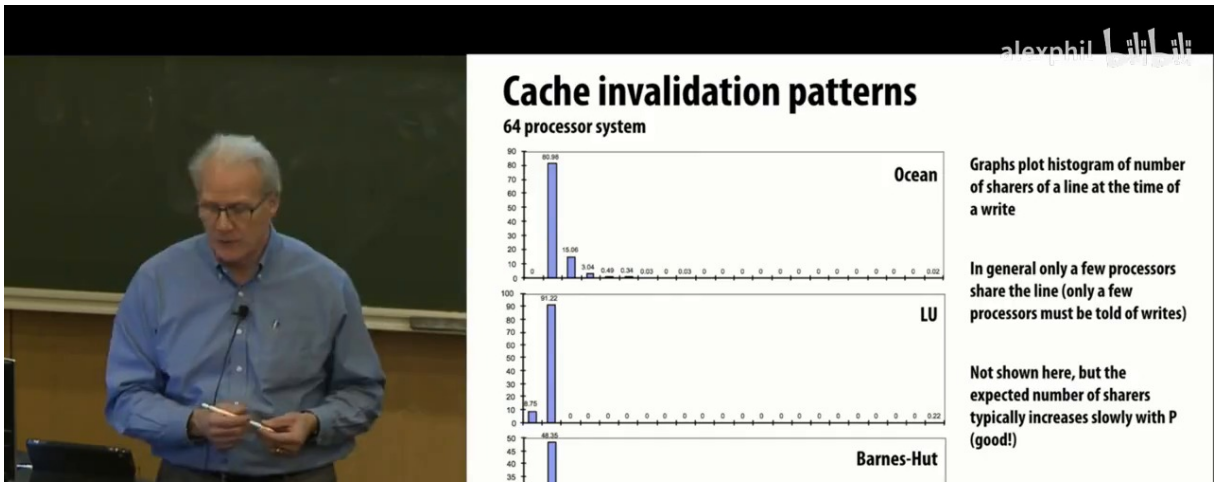


图 17: 64 处理器系统中的失效模式统计[17]

结果很有说服力：在多数程序里，90% 左右的情形只有单一副本——数值落在八成到九成之间，讲者的说法是 in the 80 to 90 percent range（讲者在此修正了自己的表述：零表示私有副本，一表示另有一份共享，需要作废）。而其他共享数的分布下降得很快。

唯一的例外是 Barnes-Hut，其中只有 48% 是单一共享。讲者解释原因：那棵树（四叉树）意味着要经常上下遍历，因此树的上层结构被大量共享。

[16] 视频画面时间区间：00:27:00–00:27:15。若为裁剪图，时间区间仍对应原视频画面。
[17] 视频画面时间区间：00:28:00–00:28:15。若为裁剪图，时间区间仍对应原视频画面。
[18] 视频画面时间区间：00:30:30–00:30:45。若为裁剪图，时间区间仍对应原视频画面。

目录的收益来自分布的形状而非平均值

如果共享者数均匀分布，目录的平均收益有限。真正带来收益的是分布严重偏斜：绝大多数情形是零或一个共享者，少数情形共享者很多。这种长尾分布让”优化常见情形”的收益远大于平均值所暗示的程度。

4.3 访问模式的分类

讲者把数据访问模式分为几类，用于定性判断各方案表现。

主要是读取的：数据很少被写。这类在任何合理方案下表现都好——各处理器都拿到干净副本，享受缓存的最大收益。

迁移型的：一处连续写一段，然后交给另一处连续读。传输本身可能有相当流量，但相对其他操作是一次性的，因此可以接受。

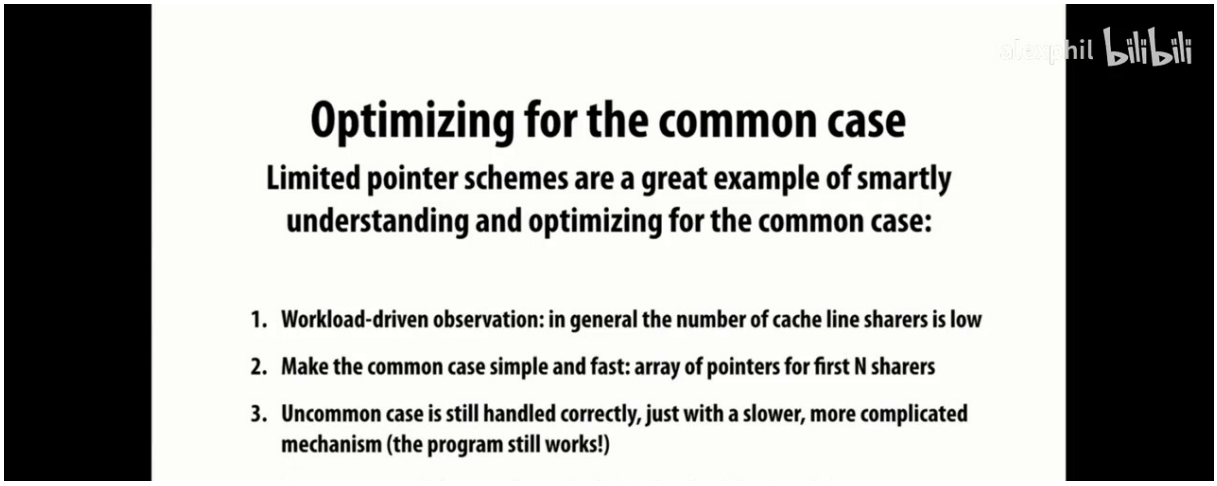


图 19: 为常见情形优化：有限指针方案是典型例子[19]

只按平均情况设计会错过优化的重点

讲者反复强调”优化常见情形”。若某类访问占八成且每次只需一条消息，那么把它做到极致，比均匀优化所有情形收益大得多。这也是有限指针、稀疏目录这些方案的设计出发点，它们都牺牲了对罕见情形的效率来换取常见情形的成本。

4.4 本章小结

目录的收益来自分布偏斜：多数应用的写只涉及零到一份其他副本，因此点对点失效远优于广播。Barnes-Hut 因四叉树上层被大量共享而成为例外。访问模式上，主要是读取的数据在任何方案下都好，迁移型数据的一次性传输相对可接受。

收益有前提，代价也有具体数字。下一章处理目录自身的存储成本。

[19] 视频画面时间区间：00:45:00–00:45:15。若为裁剪图，时间区间仍对应原视频画面。

5 目录的存储代价与压缩方案

5.1 全位向量的开销

简单的全位向量方案有一个明确代价：每个内存行都要 P 位，P 是处理器数。
整个目录的总存储可以估出来：

$$S_{\text{dir}} = \frac{M}{L} \cdot (P + 1)$$

- M：物理内存总字节数；
- L：缓存行字节数，本例为 64；
- P：处理器数量，全位向量为每个处理器一位；
- 额外的 1 是脏位。

当 P 远大于 L 的位数时，目录本身就与数据同量级，这是必须压缩的原因。
讲者算了一笔账。若 P 是 1024，每行要 1024 位说明共享状态。

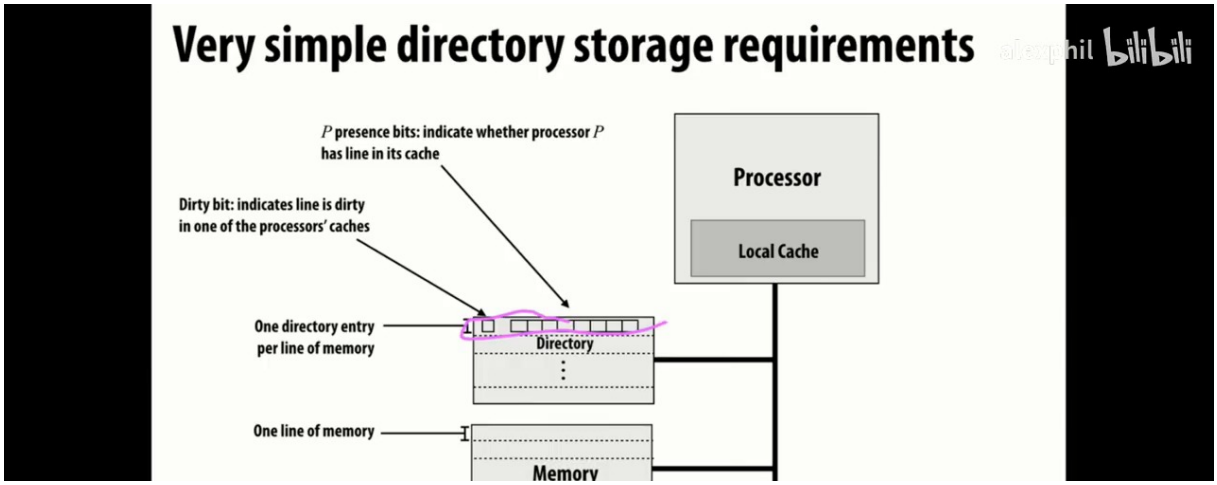


图 20: 简单目录的存储开销：每行 P 个存在位[20]

优化思路是有限指针方案：既然数据预期只出现在少数缓存里，就只存有限数量的指针。若每个指针 10 位，那么存 5 个共享者只需 50 位——远小于 1024 位。

[20] 视频画面时间区间：00:36:00–00:36:15。若为裁剪图，时间区间仍对应原视频画面。

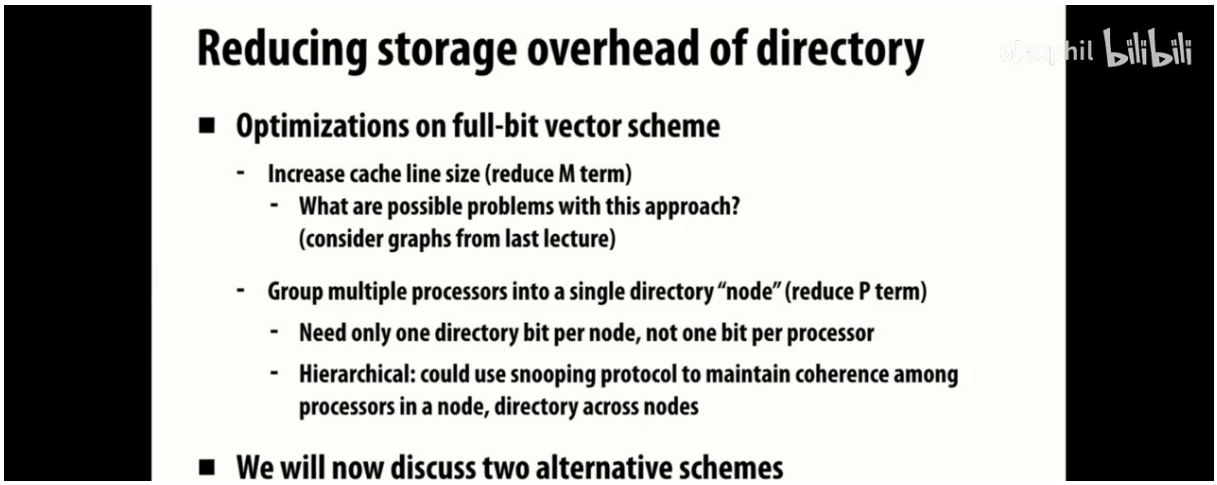


图 21: 降低目录存储开销的优化：从全位向量到有限指针[21]

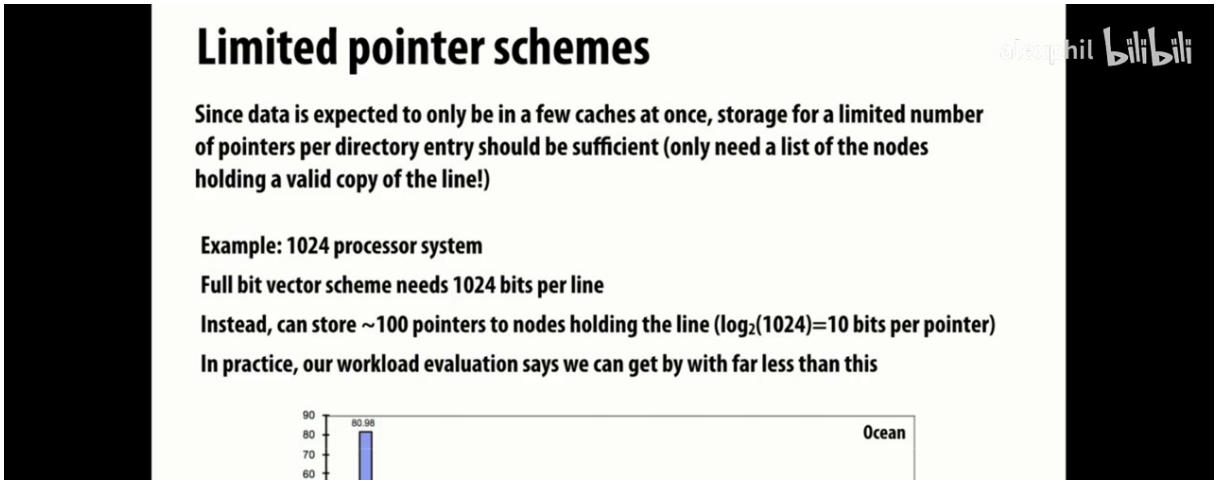


图 22: 有限指针方案：只记录少数共享者的位置[22]

$$N_{\text{bits}}^{\text{full}} = P, \qquad N_{\text{bits}}^{\text{limited}} = k \cdot \log_2 P$$

- P：处理器数量，本例中为 1024；
- k：允许记录的最大共享者数，例如 5；
- 全位向量需 1024 位，有限指针只需 $5 \times 10 = 50$ 位，约为前者的 5%。

5.2 溢出怎么办

受限方案必然遇到共享者超过 k 的情形。讲者的处理是回退到广播：声明这一行进入”无限共享者”模式，之后对该行的失效就退回广播。

[21] 视频画面时间区间：00:40:30–00:40:45。若为裁剪图，时间区间仍对应原视频画面。

[22] 视频画面时间区间：00:41:30–00:41:45。若为裁剪图，时间区间仍对应原视频画面。

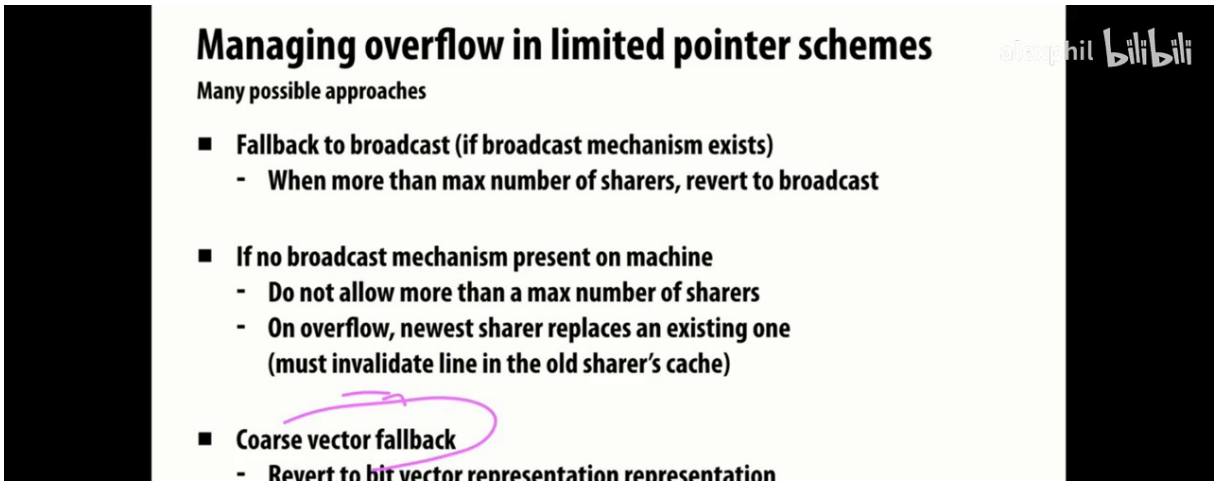


图 23: 有限指针方案的溢出处理：多种可能的回退策略[23]

他提到另一种思路：限制共享者数量本身。就像缓存满了会淘汰，目录满时可以说”我已经有最多共享者了，你得走一个”，然后向最久未使用的那个副本发失效。

溢出处理是一种刻意的容量缺失

让共享者超过上限时被迫退出，本质上是把”目录容量不足”转化为一次失效。这与缓存容量缺失同构：硬件用性能换空间。因此设计目录时必须同时考虑条目数和每条的宽度，两者都在与内存预算竞争。

5.3 稀疏目录

还有一个维度可以省：并非每一行都有共享副本。讲者说关键观察是内存中大部分内容从未被共享。

因此不必为每一行都保留目录项，只为那些确实存在共享副本的行保留即可。

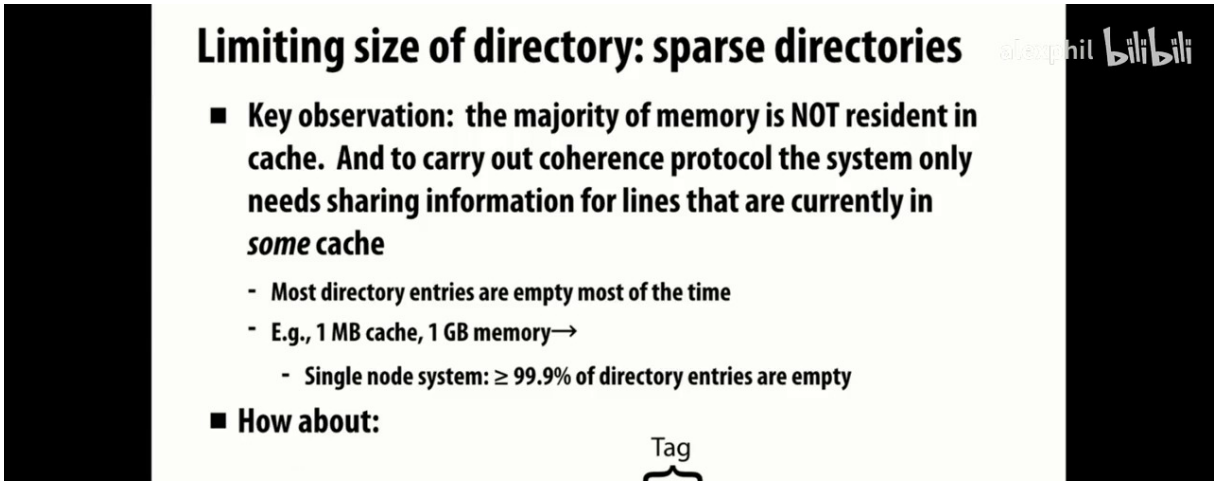


图 24: 用稀疏目录限制目录规模：只为被共享的行保留条目[24]

[23] 视频画面时间区间：00:43:30–00:43:45。若为裁剪图，时间区间仍对应原视频画面。

[24] 视频画面时间区间：00:46:00–00:46:15。若为裁剪图，时间区间仍对应原视频画面。

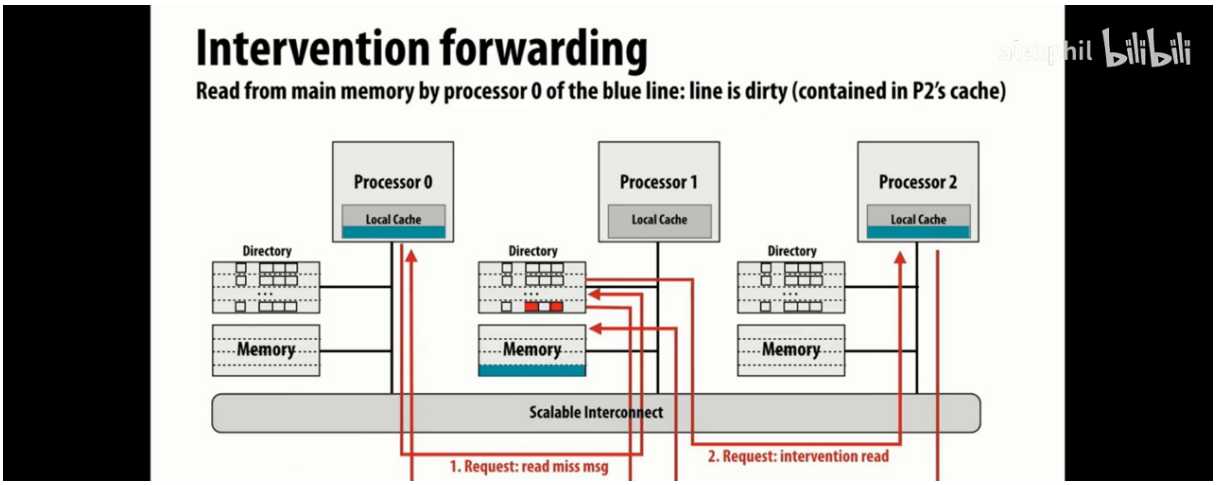


图 25: 转发介入：让持有者直接回应请求者，省去绕回 home[25]

5.4 本章小结

目录的收益来自”共享者通常很少”这一分布事实：八成到九成的写只涉及零到一份其他副本。代价是每行 P 位的存储开销，因此有全位向量、有限指针加溢出广播、稀疏目录三条压缩路径。Barnes-Hut 这类共享密集的程序是例外，其树结构上层被大量共享。

讲者也说明这种代价收益权衡是所有硬件设计共同的思维方式：先弄清典型工作负载，把容易且重要的情况做好。

6 真实机器的实现与优化

6.1 转发介入

讲者介绍了一个把读延迟降低的优化。回顾”读缺失但行为脏”的情形：原本是 home 把请求转给持有者，持有者再把数据送回 home，home 再给请求者——绕了一圈。

转发介入的做法是让持有者直接把数据回给请求者，省掉中间的往返。

6.2 Intel Core i7：L3 充当目录

讲者展示了真实处理器中的做法：Core i7 的共享 L3 缓存充当集中的目录。

[25] 视频画面时间区间：00:48:00–00:48:15。若为裁剪图，时间区间仍对应原视频画面。

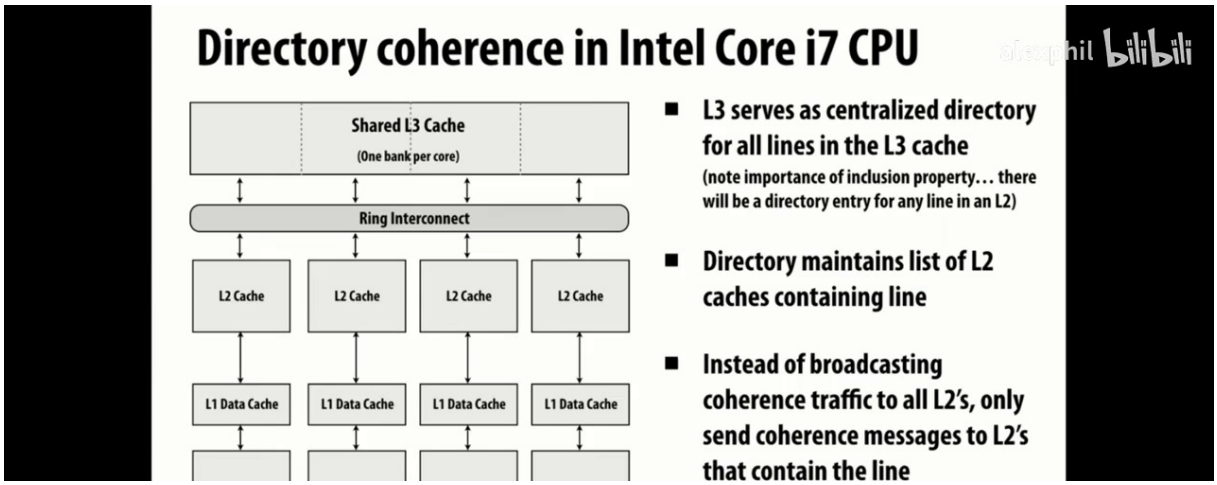


图 26: Intel Core i7 中的目录式一致性：L3 作为集中目录[26]

6.3 多插槽系统

多插槽 Intel 系统里，一致性要跨插槽完成，内存侧的目录与插槽间互连共同承担这件事。

[26] 视频画面时间区间：00:49:00–00:49:15。若为裁剪图，时间区间仍对应原视频画面。

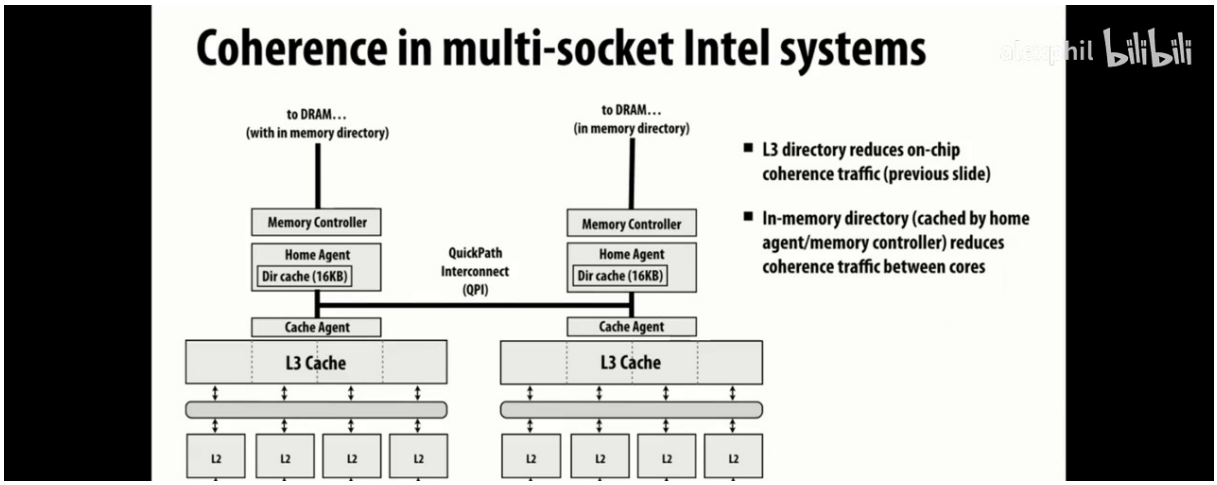


图 27: 多插槽 Intel 系统中的一致性[27]

6.4 Xeon Phi

讲者最后介绍了一类很不同的处理器：Xeon Phi，是 Intel 试图与 NVIDIA 争夺高性能计算的产物。

它的定位是”众核”系统，单芯片上有大量 x86 核心，并为它们提供一致的缓存内存接口。课程机器上挂着 14 块这种卡。讲者说他们能用的版本略旧，代号 Knight's Corner。

他顺带提到一个有意思的事实：天河二号就是用这类芯片搭起来的，曾多年位居世界最快计算机，现在是第二或第三。

讲者随后说明了它的互连方式，这一点值得记住，因为它把前面两种方案结合了起来。

这些处理器之间用一条环形网络通信，而这条环同时被用作维持缓存一致性的基础。内存物理上被划分给不同的内存控制器（大约四个或八个），这些控制器可以看作环上的节点。消息沿环流动，在缓存控制器与内存控制器之间传递。

因为全都在一颗芯片上，环可以做得很宽：数据总线是 64 字节宽，而不是 64 位。这让带宽很高、延迟很低。

环的好处是可以灵活地广播与定向：发一条消息绕环一整圈就实现了广播；若目标是附近的节点，消息只需走到能到达的地方就停。而且它是双向的，两个方向都能发。

于是协议在形式上又回到了前面看过的那种全局协议：发出”我需要一份副本”或”你必须作废你的副本”这类消息。作废消息绕环一整圈，请求消息则走到附近的持有者即可。

[27] 视频画面时间区间：00:51:00–00:51:15。若为裁剪图，时间区间仍对应原视频画面。

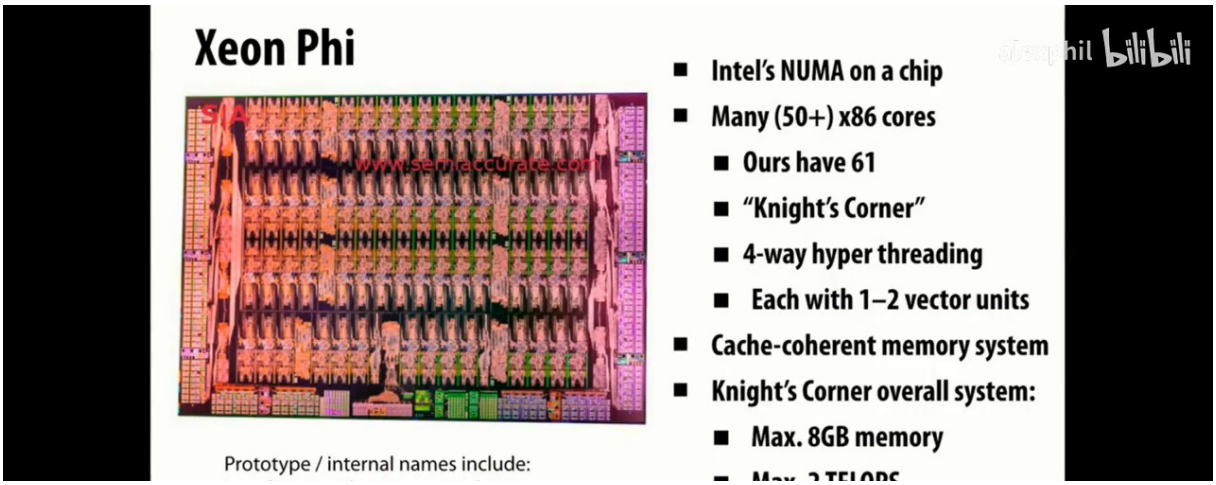


图 28: Xeon Phi：片上 NUMA，50 个以上 x86 核心与 4 路超线程[28]

层次化是真实系统的共同答案

从双插槽到 Xeon Phi，一致性都不是靠单一机制解决的，而是多层叠加：核内缓存、插槽内共享缓存、插槽间目录、片上分区内存。每层处理自己范围内的通信，只有跨层时才付出更高代价。这解释了为什么讲者说现实中没人用简单版本。

6.5 本章小结

真实系统在目录基础上加入了转发介入（降低读延迟）、集中目录（i7 的 L3）、跨插槽目录与片上 NUMA（Xeon Phi）。共同点是层次化：让大多数通信在局部完成，只有必要时才跨越更高层。

讲者在此结束了硬件部分的讨论。接下来是这一讲最容易被忽视、也最容易让程序出错的部分：内存一致性模型。

7 内存一致性模型：一致性没解决的那一半

讲者在最后转向一个与前面完全不同的问题，他明确说这是本讲对程序员最重要的部分之一。

7.1 缓存一致性不等于程序直觉

前面所有的讨论都在保证一件事：对同一个地址，所有处理器看到一致的顺序。但这并不保证不同地址之间的操作顺序符合程序员的直觉。

考虑两个线程：线程 A 写数据，然后置一个标志位表示”数据已就绪”；线程 B 循环等待标志位，看到后就读取数据。

从程序员角度看这个模式天经地义。但在允许重排的硬件上，A 的两条写可能在别人看来是反序的——标志位先可见、数据后可见。于是 B 看到标志却读到旧数据。

[28] 视频画面时间区间：00:55:30–00:55:45。若为裁剪图，时间区间仍对应原视频画面。

一致性的作用域是单一地址，不是跨地址的顺序

缓存一致性只保证同一地址上的写对所有观察者有序。至于”先写 X 再写 Y”这个顺序是否对别人可见，属于内存一致性模型的范畴。前者是所有共享内存机器的必备，后者因架构而异，可以从严格顺序一路放宽到几乎不保证。

7.2 为什么会允许重排

允许重排是为了性能。顺序执行所有内存操作意味着每一次访存都要等上一次完成，这对流水线与多级缓存都是巨大浪费。

放宽模型让硬件可以把写放入缓冲、可以先执行后续不相关的读、可以让不同位置的写并行完成。代价是把”保持顺序”的责任部分转移给了程序员。

此外还有第二个重排来源：编译器。即使硬件严格保序，编译器也可能为了优化而调换两条不相关的访存语句。因此单靠硬件保序不足以让直觉成立。

双重检查锁一类的模式在弱模型上会失效

这类模式依赖”先构造对象再发布引用”的顺序，而顺序可能被硬件或编译器破坏。正确做法是使用语言提供的内存序原语：写侧用释放语义发布，读侧用获取语义接收，或在初始化时使用原子操作加栅栏。只加锁访问一部分字段而让另一部分裸露，是典型误用。

7.3 怎么写出正确的代码

讲者给出的方向是原则性的而非具体的原语清单：不要依赖自己没验证过的顺序假设。

实用规则有两条。第一，跨线程共享的数据要么全部通过同步原语访问，要么用带有明确内存序语义的原子操作。第二，任何”先准备数据再发出信号”的模式，都必须用释放与获取语义把两侧配对起来，让硬件和编译器都清楚这个顺序不能被打破。

顺序的建立需要两侧配合

只在一侧加栅栏不足以建立跨线程的先后关系：发布方需要用释放语义确保之前的写先可见，接收方需要用获取语义确保之后的读不会提前。两侧缺一，顺序就可能被破坏。这与锁的配对使用是同一个道理。

7.4 布局是最后一项可调的手段

讲者在结尾提到，当人们真正开始调优程序时，会去看数据布局、访问模式这些条件，并可以通过各种提示帮助编译器与硬件。

他同时提醒这不总是可靠，也不容易做到。这一点与前面的伪共享、分块形状、目录共享分布互相呼应：性能的决定因素往往在数据怎么摆，而不在算法怎么写。

7.5 本章小结

缓存一致性保证单一地址的顺序，内存一致性模型才决定跨地址的顺序。允许重排是为性能付出的代价，而重排可能来自硬件也可能来自编译器。因此跨线程的通信必须借助带序语义的原语配对使用，不能依赖”写的先后看起来应该如此”。

8 总结与延伸

这一讲补上了第 11 讲没讲完的一块，然后把话题推向协议实现与程序语义两侧。

伪共享是最容易被忽视的一种人为通信：数据落在同一缓存行、被不同核使用，逻辑上毫无冲突的代码会在协议层面互相作废，实测可以差两倍以上。对策是填充或改变布局，让共享只发生在边界。这也解释了第 8 讲分块布局选择的动因。

监听协议的瓶颈是广播：流量随参与者增长，超过约半打缓存后互连成为中心瓶颈。层次化监听把局部流量吸收在低层，但树根仍可能成为瓶颈。目录用位向量记录每行的持有者，把通信变成点对点；实测显示八成到九成的写只涉及零到一份其他副本，这正是目录收益的来源。代价是每行 P 位的存储，因此有有限指针加广播回退、稀疏目录等压缩方案，Barnes-Hut 这类树结构程序则是共享密集的反例。真实系统在此基础上加了转发介入、集中目录、跨插槽目录与片上环网，共同点是层次化。

最后，缓存一致性只保证单一地址的顺序；跨地址的顺序属于内存一致性模型，可以被放宽以换取性能，而重排可能来自硬件也可能来自编译器。因此跨线程的通信必须靠带序语义的原语配对完成。

本讲义在讲者材料之上做的压缩，有四点需要与讲者原话区分：

- 伪共享与真正的共享是两个不同的性能问题。前者因为同一缓存行而互相干扰，后者因为访问同一变量而需要同步。前者的解法是改布局（不改语义），后者的解法是改协议（加锁或原子操作）。混用两种解法不会有效。
- 目录的价值来自分布的形状。若共享者数均匀分布，全位向量与有限指针的差距有限；正因为现实中严重偏斜，压缩方案才有意义。评估任何”优化常见情形”的技术，都要先确认分布确实偏斜。
- 溢出与稀疏化都是把硬件资源压力转化为性能损失。它们不改变正确性，只改变退化方式。理解这一点，才能在容量与性能之间做出有依据的取舍。
- 一致性模型的责任划分是被刻意设计的。硬件放宽顺序是为了性能，代价是程序员必须显式表达依赖。因此”我的代码在单核上是对的”从来不构成它在多核上正确的理由。

这堂课的边界在于它仍是导论：讲者多次表示真实协议细节极其复杂，内存模型的完整规则需要专门讨论。因此本文给出的是机制与陷阱的地图，不含可照抄的协议实现。

两个可以直接从本讲数据提出的下一步问题：其一，目录的存储开销与共享者上限之间存在直接的权衡，那么在什么工作负载下应当选择全位向量而非有限指针？其二，内存一致性模型允许的重排范围因架构而异，若要写一份既正确又能在多种架构上保持性能的代码，应当采用最严格的模型还是最弱的模型，判据是什么？